

COST & PERFORMANCE ADVISORY

Terraform Execution: Bottleneck & Optimization Plan

Why the backend's Terraform-execution design can't move to Lambda as-is, what it's currently costing to run, and a concrete recommendation for where to spend engineering effort next.

Companion document to *InfraFlow Frontend Architecture*. Figures in this document (disk usage, folder counts, cache sizes) were measured directly from the running `IAAS backend/.terraform-runs` directory at the time of writing — they will drift as the system keeps running, and should be re-measured before acting on any number here that looks stale.

Table of Contents

1. The Bottleneck, Stated Plainly
2. Root Cause: An Architecture Mismatch
3. Current State — What's Actually on Disk Right Now
4. Cost Comparison: EC2 vs. Lambda vs. CodeBuild/Fargate
5. Recommended Architecture
6. Decision Matrix — When to Revisit This
7. Concrete Next Steps

SECTION 1

The Bottleneck, Stated Plainly

The backend's Terraform-execution path (`terraformDeploymentRunner.js`) spawns the real `terraform` binary as a child process and writes its working files — generated `main.tf`, downloaded provider plugins, and the deployment's `terraform.tfstate` — straight to local disk. That design is correct and simple *as long as it runs on a persistent server with persistent disk*. It breaks the moment you try to move it to Lambda or any other stateless/ephemeral compute, because:

- Lambda's filesystem is wiped between cold starts — a freshly-created `terraform.tfstate` file would vanish, and with it AWS's only record of what real infrastructure was just created (the resources themselves stay in AWS; Terraform just loses the ability to track or destroy them).
- A real `terraform apply` can run longer than Lambda's 15-minute hard cap.
- Without a persistent local cache, the ~700MB AWS provider binary would need re-downloading on every cold invocation.

THIS IS NOT A HYPOTHETICAL — IT ALREADY CAUSED A REAL INCIDENT

Earlier in this project, someone zipped the entire `IAAS backend` directory (instead of just Lambda source code) to use as a Lambda deployment package. It came out to **606 MB compressed / ~3 GB uncompressed**, because it accidentally swept up multiple full copies of the Terraform working directories described in this document — including Terraform state files and, in the copies sampled, the AWS provider binary. That's a direct, concrete illustration of why this directory's design doesn't travel well: it's easy to accidentally include, expensive when you do, and functionally meaningless outside the machine that created it.

SECTION 2

Root Cause: An Architecture Mismatch

The underlying issue is that **two very different kinds of work are currently running on the same compute**, under the same deployment target, with the same code:

Workload	Shape	Best-suited compute
The HTTP API (auth, CRUD on diagrams/deployments/tickets, status polling)	Short-lived, bursty, stateless between requests	Serverless (Lambda) — genuinely a good fit, and what was built in the companion package this week
<code>terraform init / plan / apply / destroy</code>	Long-running (minutes), needs persistent local disk for state + provider cache, no natural request/response shape	A persistent or long-running compute target: EC2, ECS/Fargate, or a build service like CodeBuild — not Lambda

Trying to force both onto Lambda is what creates the bottleneck. The fix isn't "avoid Lambda entirely" or "avoid serverless entirely" — it's recognizing these are two different jobs that should be allowed to live on two different kinds of compute, coordinated through a durable, shared state store (S3 + DynamoDB) instead of a local folder.

Current State — What's Actually on Disk Right Now

Measured directly from `IAAS backend/.terraform-runs` at the time of writing:

TOTAL SIZE

6.1 GB

DEPLOYMENT FOLDERS

117

SHARED PLUGIN CACHE

705 MB

Doing the arithmetic: $6.1 \text{ GB} - 705 \text{ MB} \approx 5.4 \text{ GB}$ spread across 117 per-deployment folders, or roughly **~46 MB** per folder on average. That's a reasonable size for a state file, generated `main.tf`, lock file, and logs — **it is not each folder holding its own full copy of the ~700MB provider binary**. This confirms the existing shared plugin cache (`TF_PLUGIN_CACHE_DIR` , implemented via `getPluginCacheDir()`) is doing its job on average.

THE REAL RISK ISN'T DUPLICATION — IT'S UNBOUNDED RETENTION

117 deployment folders exist and, as far as the current code shows, none are ever automatically removed — including folders for deployments that were later destroyed or cancelled. Provider binaries are deliberately kept after a destroy (the code only removes the `.terraform` provider cache post-destroy, keeping `main.tf` /state for audit purposes — a deliberate, reasonable choice). But nothing currently bounds how many of these folders accumulate over the life of the system. At today's **~46 MB/deployment** average, 1,000 deployments would be ~46 GB; 10,000 would be ~460 GB. This is the actual long-term capacity risk on whatever disk this runs on — not a one-time incident, but a slow leak with no drain.

NOT ACTED ON IN THIS DOCUMENT

This document only measures and recommends — no cleanup was performed. A retention policy (e.g. "archive to S3 and delete local folders for deployments in a terminal state older than N days") is listed as a next step in Section 7, not implemented here, since it touches real deployment state and shouldn't be automated without an explicit decision on the retention window.

Cost Comparison: EC2 vs. Lambda vs. CodeBuild/Fargate

4.1 Running the Terraform-execution workload specifically

Approach	Cost pattern	Est. monthly cost	Fit for this workload
EC2 (t3.small, 24/7) + EBS for plugin cache	Fixed, independent of usage	~\$17–20 (instance) + ~\$2–5 (EBS) ≈ \$20–25/mo	Best match — persistent disk, no timeout, matches existing code as-is
EC2 (t3.medium, 24/7)	Fixed	~\$30 (instance) + ~\$5 (EBS) ≈ \$35/mo	Same, with more headroom for concurrent applies
AWS CodeBuild (pay-per-run)	Variable, scales with usage	~\$1.50 per 100 applies (general1.small, ~3 min/apply)	Good fit if usage is bursty/low-volume; needs S3-backed plugin cache to avoid re-download cost per build
AWS Lambda	Variable, but structurally mismatched	Cheap per-invocation, but requires re-architecting around the 15-min cap and ephemeral disk	Poor fit — the underlying constraints (Section 1–2), not the price, are the blocker

4.2 Running just the API layer (auth, CRUD, polling)

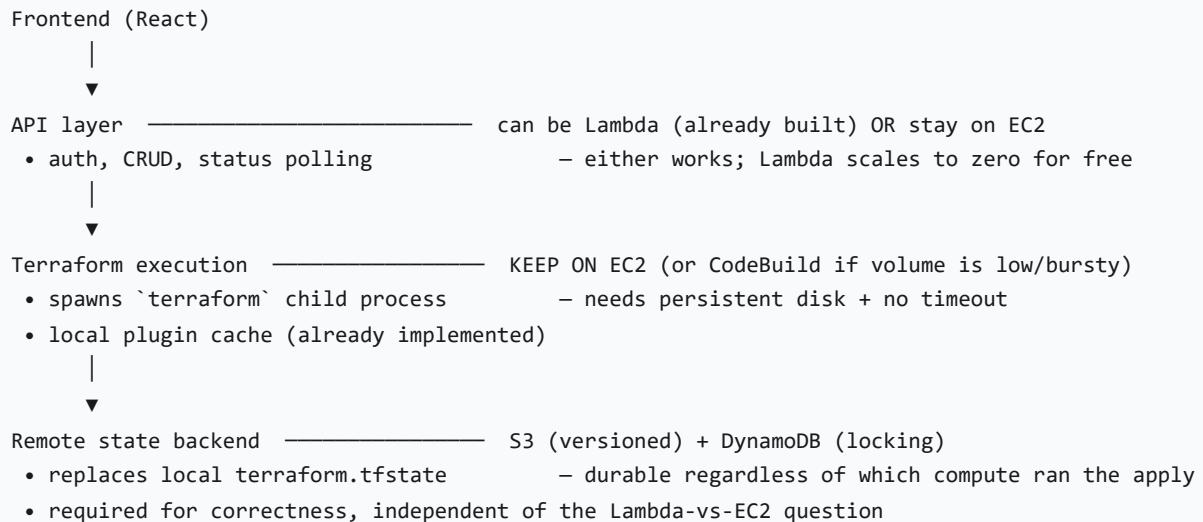
Component	Est. monthly cost
Lambda (the package already built this week)	~\$2–\$10 at low-moderate traffic, largely inside AWS's free tier
API Gateway (HTTP API tier, if fronting Lambda)	~\$1 per million requests

The API layer is genuinely cheap and well-suited to Lambda regardless of what happens with the execution layer — this part of the earlier serverless work is not wasted either way.

4.3 Crossover point

At **low deployment volume** (a handful to tens of applies/month), pay-per-use compute (CodeBuild) is cheaper because you pay near-\$0 when idle. At **moderate-to-high volume**, or when applies run long, EC2's flat **\$20–35/month** becomes cheaper *and* gives you the plugin-cache/no-timeout benefits for free. Given the current system has produced 117 deployments over its life so far, actual volume is almost certainly still in the "EC2 wins on both cost and simplicity" zone — see Section 6 for when that could change.

Recommended Architecture



5.1 What changes vs. today

- **State backend:** migrate from local `terraform.tfstate` files to an S3+DynamoDB remote backend. This is worth doing regardless of the EC2/Lambda decision — it's what makes state durable, team-safe, and recoverable if the EC2 instance is ever replaced.
- **Execution compute:** keep it on EC2 (no code change required — this is what the code already assumes). Revisit CodeBuild/Fargate only if usage volume grows enough to make the crossover in Section 4.3 favor it.
- **API compute:** optional — the Lambda package already built this week can front the API if scale-to-zero is valuable, or the API can simply stay on the same EC2 instance as execution (simplest, one deployment target, no split to maintain).
- **Retention:** introduce an explicit cleanup policy for `.terraform-runs/` folders belonging to deployments in a terminal state (destroyed/cancelled) past some age — see Section 7.

SECTION 6

Decision Matrix — When to Revisit This

Signal	What it means	Action
Deployment volume regularly exceeds ~300–500 applies/month	You're past the EC2/CodeBuild cost crossover point	Re-evaluate CodeBuild or Fargate for execution; keep the S3-backed plugin cache either way
Concurrent deployments start queueing/blocking each other on the single EC2 instance	Single-instance execution is now a throughput bottleneck, not just a cost question	Move to Fargate (parallel tasks) or a CodeBuild project with concurrency, both reading/writing the shared S3+DynamoDB state backend
<code>.terraform-runs</code> disk usage approaches the EC2 instance's EBS volume size	Retention policy is overdue	Implement the archive-then-delete policy from Section 7 before it becomes an outage
API traffic (not execution) becomes the larger cost line item	Unlikely given today's numbers, but possible at high user-count scale	This is where Lambda's pay-per-request model actually starts mattering — move just the API layer

Concrete Next Steps

1. **Migrate Terraform state to S3 + DynamoDB.** Highest-value, lowest-risk change — improves durability and team-safety immediately, and is a prerequisite for ever running execution anywhere other than exactly one fixed EC2 instance.
2. **Leave execution on EC2 for now.** No code change required; it's already the right fit per Section 4. Don't spend engineering time moving this to Lambda unless a Section 6 signal actually fires.
3. **Decide and implement a retention policy for `.terraform-runs/`.** Not implemented in this document — needs an explicit decision on retention window (e.g. 30/90 days post-destroy) and whether to archive deleted folders to S3 first for audit purposes before deleting locally.
4. **Optional: front the API with the Lambda package already built.** Low-risk, low-cost, independent of the execution-layer decision — do this only if scale-to-zero for the API specifically is valuable, not as a step toward moving execution to Lambda too.
5. **Re-measure before acting.** The 6.1 GB / 117-folder figures in this document are a point-in-time snapshot — confirm current numbers before sizing any EBS volume or setting a retention window.

Companion to *InfraFlow Frontend Architecture*. Both documents reflect a point-in-time investigation and should be re-verified against the running system before being relied on for capacity or budget decisions.